

THE UNIVERSITY OF CHICAGO

MASTERS IN COMPUTATIONAL SOCIAL SCIENCE
THE UNIVERSITY OF CHICAGO

MACS 30111

Working with Files

Agenda/Misc

- **Files for today: sync your fork, then open in_class_exercises/M4_files/**
  - M3b_files_in_class.ipynb: Part 1, the buildings exercise (Part 2, Nobel Prizes, is M3b_files_nobel.ipynb)
  - Data: names.txt, instructors.csv, nobelPrizes.json
- **Reading:** CS with Applications §4.1, Working with Files
- Survey results
- PA: timing and process
- Quiz
- Files

Survey says

- Most people feel prepared
- Different people like scripts vs jupyter notebook (even split!)
- Pace: a little too fast
- Fewer class examples, more lecture (but scripts/notebook most helpful part)

Survey says

- **Can fix this year:**
  - Quiz timing: more time
- **Can fix next year:**
  - Slower start to the course with more on-boarding
- **Thinking through:**
  - Interactive examples – more scaffolding

Data types recap

- **Lists**: mutable, mix and match types
- **Tuples**: immutable, CAN mix and match types
- **Dictionaries**: key/value pairs
- **Sets**: groupings (present/not), cannot ‘index’

Topics:

- Basic file I/O
  - Open: load the data from disk
  - Read: manipulate the data
  - Close: print the results or write the data back to disk
- Working with tabular data using CSV files
- Working with JSON files
- Other file formats

Common Programming Pattern

Common pattern when working with data:

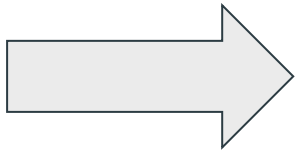
1. Read the contents of a file (or files) from disk and load the data into one or more data structures
2. Manipulate the data in some way
3. Print the result or write the data back to disk

Sample application

Given a file of email addresses (username@domain), construct a file with the corresponding user names.

`instructor-email.txt`

"email"
"hdambane@uchicago.edu"
"dapeterson@uchicago.edu"
"zwang13@uchicago.edu"
"jclindaniel@uchicago.edu"
"nardin@uchicago.edu"



`instructor-email-short.txt`

hdambane
dapeterson
zwang13
jclindaniel
nardin

Coding practice: 4.1.1

Basic File I/O

To access the contents of a file, we first need to open it:

```
f = open("instructor-email.txt")
```

file pointer

To read data from a file, we use the read method:

```
addrs = f.read()
```

read the entire contents into a string

When we are done with a file, we need to close it:

```
f.close()
```

close the file pointer

Coding practice: 4.1.1

Alternative to *close()*

The with statement to ensure that a file is closed once we're done with it:

```
with open('instructor-email.txt') as f:
    s = f.read()
    email_addresses = sorted(s.split())
```

Coding practice: 4.1.1

Predict: what prints?

- Run it in your head first. Write down your answer, then run it.

```
with open("names.txt") as f:
    first = f.read()
    second = f.read()

print(len(first) > 0)
print(repr(second))
```

- Turn to a neighbor: why is second what it is? What would you do to read the file again?

Reading: §4.1.1 Basic file I/O

Read the file one line at a time

Use a for loop to iterate over a text file line by line:

```
with open('instructor-email.txt') as f:
    for line in f:
        print(line)
```

extra empty line

```
with open('instructor-email.txt') as f:
    for line in f.readlines():
        print(line)
```

line.strip()

Coding practice: 4.1.1

Write data to a file

To write to a file, we must open the file in write mode.

```
with open("names.txt", "w") as f:
    f.write("Anne Rogers\n")
    f.write("Borja Sotomayor\n")
    f.write("Yanjing Li\n")
    f.write("Matthew Wachs\n")
    f.write("Todd Dupont\n")
```

We can also use print to avoid having to worry about the newline.

```
with open("names2.txt", "w") as f:
    print("Anne Rogers", file=f)
    print("Borja Sotomayor", file=f)
    print("Yanjing Li", file=f)
    print("Matthew Wachs", file=f)
    print("Todd Dupont", file=f)
```

Very important:

- Opening an existing file in write mode will **wipe all its contents**!
- Opening a file that does not exist in write mode will **create** the file.

Coding practice: 4.1.2

Different ‘modes’

- You can open files in different ‘modes’
  - r: ‘read’ mode (default)
  - w: ‘write’ mode (needs specified)
  - a: append

If you’re just reading a file, you can operate as normal. If you’re wanting to write a new file, *then* you will use “w”.

DANGER ALERT!!! In “w” mode, you will OVERWRITE THE PREEXISTING FILE!!

Function to trim (can give issues re: writing output)

```
def strip_domain(input_filename, output_filename):
    '''

    Strip the domain names off the email address from the input
    file and write the resulting usernames to the output file.

    Inputs:
      input_filename: (string) name of a file with email addresses
      output_filename: (string) name for the output file.
    '''

    # Load data into a data structure (a list of strings)
    email_addresses = []
    with open(input_filename) as f:
        for line in f:
            email = line.strip()
            email_addresses.append(email)
```

```
#(cont'd)

    # Transform the data
    usernames = []
    for email in email_addresses:
        username, domain = email.split("@")
        usernames.append(username)

    # Write the data
    with open(output_filename, "w") as f:
        for username in usernames:
            print(username, file=f)
```

Summary

The common programming pattern:

1. Load the data from disk:
    a. Open a file to read
    b. Read the contents of the file from disk
    c. Load the data into a data structure
2. Manipulate the data in some way
3. Print the result or write the data back to disk
    a. Write the data
    b. Close the file (or use a with statement when you open it)

Coding practice: 4.1.2

CSV (Comma Separated Values) format

CSV files are useful for storing tabular data: any data that can be organized into rows, each with the same columns (or "fields")

`instructors.csv`

header

```
id,lname,fname,email
hdambane,Dambanemuya,Henry,hdambane@uchicago.edu
dapeterson,Peterson,David,dapeterson@uchicago.edu
zwang13,Wang,Zhao,zwang13@uchicago.edu
jclindaniel,Clindaniel,Jon,jclindaniel@uchicago.edu
nardin,Nardin,Sabrina,nardin@uchicago.edu
```

Read file using *csv* module

- csv.DictReader - read rows from a CSV file into dictionaries
- csv.DictWriter - write dictionaries into rows of a CSV file

Alternatively, we could also use:

- csv.reader - read rows from a CSV file into a list of lists
- csv.writer - write lists into rows of a CSV file

Applied practice!

Working with text

What if you want to skip lines?

- Multiple ways to approach:

```
new_list = []
with open('names.txt') as names:
    next(names)
    next(names)
    next(names)
    new_list = names.readlines()
```

```
new_list = []
with open('names.txt') as names:
    for i, line in enumerate(names):
        if i > 3 and i < 149:
            new_list.append(line)
```

Exercise

- You are working on a project creating a directory of buildings based on where MACSS classes typically meet.
- Use text file: in_class_exercises/M4_files/names.txt
- Work through **Part 1** of M3b_files_in_class.ipynb with a partner
- How would you load your data?
- Next, you want to select the following buildings: sched = ["1155", "SS", "TTI", "K", "STU"]
- and output the new list into a separate file
- **Reading:** §4.1.1 Basic file I/O and §4.1.2 Writing data to a file

Predict: how many buildings match?

- Five buildings in sched. How many does this find? Vote: 0, 1, 5, or an error.

```
sched = ["1155", "SS", "TTI", "K", "STU"]
matches = []
with open("names.txt") as f:
    for line in f:
        abbrev = line.split("\t", 1)[0]
        if abbrev in sched:
            matches.append(abbrev)
print(matches)
```

- Then run it (notebook Part 1, Step 4). Surprised? Try print(repr(line)).

Bringing the exercise together:

```
# making it pretty:

def get_buildings(input_filename, output_filename, sched):
'''

extract relevant buildings from campus list

Inputs:

input_filename: (string) name of a file with buildings

output_filename: (string) name for the output file.
'''
```

Discuss: this version tests line[0] in sched with no .strip() on the abbreviation, and it finds all five. Why?

```
# Load data into a data structure (a list of strings)
buildings = []
with open(input_filename) as f:
        for line in f:
                builds = line.strip().split("\t", 1)
                buildings.append(builds)

# Transform the data
buildings_select = []
for line in buildings:
        if line[0] in sched:
                buildings_select.append(line)

# Write the data
with open(output_filename, "w") as f:
        for build in buildings_select:
                print(build, file=f)
```

Exam prep: spot 3 errors and rewrite code

```
# making it pretty:


def get_buildings(input_filename, output_filename, sched):
'''


extract relevant buildings from campus list


Inputs:

input_filename: (string) name of a file with buildings

output_filename: (string) name for the output file.
'''
```

```
# Load data into a data structure (a list of strings)
buildings = []
with open(input_filename) as f:
        for line in f:
                builds = line.strip().split("\t", 1)
                buildings.append(builds)

# Transform the data
buildings_select = []
for line[0] in buildings:
        if line in sched:
                buildings_select.append(line)

# Write the data
with open(output_filename) as f:
        for build in buildings:
                print(build, file=f)
```

Exam prep: the 3 errors

1. for line[0] in buildings:
   - The loop variable must be a name: for line in buildings:, then test line[0] in sched
2. open(output_filename)
   - Default mode is read, so print(..., file=f) fails. Use open(output_filename, "w")
3. for build in buildings:
   - Writes every building. Loop over buildings_select

Check your fix in the notebook's exam prep cell. Which of these errors would Python catch for you, and which would just give a wrong answer?

JSON (JavaScript Object Notation) format

JSON is a lightweight data-interchange / data-storage format commonly used in web services.

Supports different types:

- Object: key-value pairs separated by commas
  - Keys must be strings
  - Values must be valid JSON data types
- Array: empty list or list of objects
- Value: string, number, object, array, true, false, null

File operation using JSON module

String operation:

- json.dumps: encodes data into JSON format string
- json.loads: decodes JSON format string into a data structure

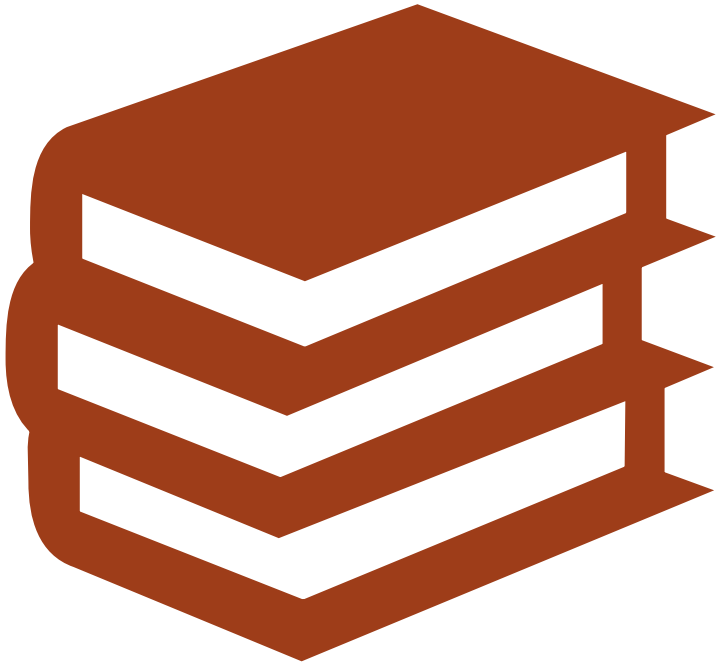
File operation:

- json.dump: encodes data into a JSON file
- json.load: decodes data from a JSON file into a data structure

Coding practice: 4.1.4

Prep for Tuesday (10/27)

- Post on ED:
  - One question you have (be specific, include an example)
  - One question you think would be good for the midterm
- Finish the buildings notebook (Part 1). Solutions are posted after class.
- Midterm: Thursday 10/29, in class



Additional practice

Application

- You are a social scientist interested in who has won the Nobel Prize.
- In your groups, work through **Part 2** (M3b_files_nobel.ipynb). The data is in in_class_exercises/M4_files/nobelPrizes.json
- Use reading §4.1.4 JSON to load and inspect the data
- Inspect before you loop: what type is it? What are the keys? This is a tiny slice of the data, so it won't overwhelm your machine.
- Provide a dictionary of the number of years and awards covered by this dataset.
- **Bonus**! Export this to a json file with json.dump, then load it back and compare